Verifiable Computation Under Embedded Constraints: Architectures, Energy Trade-offs, and Experimental Observations

Michael Tass MacDonald
Baramay Station Research Inc.
Stony Rapids, Saskatchewan, Canada
michael@baramaystationresearchinc.ca

January 2025

Abstract

Verifiable computation has emerged as an important enabling technology for secure and auditable computing, yet its practical deployment on energy- and latency-constrained hardware remains limited by the overheads associated with execution logging and verification. This article reviews recent developments in transparent verification and zero-knowledge proof systems from the perspective of embedded and edge computing, with emphasis on physical constraints such as power consumption, memory bandwidth, and real-time operation. As an experimental case study to ground the discussion, we examine an embedded witness-recording pipeline designed to support downstream STARK-style verification workflows.

The evaluated system couples a delta-encoded witness-recording mechanism with cryptographic integrity binding and was implemented on a Raspberry Pi 5 equipped with a Hailo-8 neural processing unit. Under the reported test configuration, the recorder sustained approximately 11.47 million recorded steps per second, with per-step recording latencies on the order of tens of nanoseconds. The associated computation kernel—a Lyapunov-stable linear contraction operator defined in PyTorch, exported via ONNX, and compiled to Hailo-8 silicon—exhibited measured execution rates of 49,859 operations per second in hardware-only mode and 28,394 operations per second when invoked through a Python-mediated runtime.

Witness data were captured using Cython-optimized routines with delta encoding and multicore sharding, producing approximately 1.4 GB of compressed shard data for 500 million recorded steps, corresponding to an observed compression ratio of roughly 14× under the tested workload. Integrity binding was achieved via cryptographic commitment roots validated through recomputation and contract-level checks, with integration demonstrated using Cairo smart contracts on StarkNet. Throughput-derived frame-budget estimates, along with energy and cost calculations based on stated power-accounting assumptions, are presented as contextual indicators rather than general performance bounds.

All implementation artifacts, execution logs, and validation results are publicly released to support independent inspection and reproduction. The reported measurements characterize the behavior of a specific witness-recording architecture under a constrained workload and hardware configuration, and are intended to inform broader discussions of verifiable computation for embedded systems rather than to establish general claims about end-to-end zero-knowledge proof generation performance.

Code Availability: Complete implementation, validation logs, and reproducibility instructions available at https://github.com/Baramay-Station/TetraKlein-Unified-Architecture-Whitepaper

1 Introduction

Zero-knowledge proof systems have emerged as a fundamental primitive for verifiable computation, enabling cryptographic verification of arbitrary computations without revealing underlying data [1, 2, 6]. However, practical deployment remains challenging: in many existing systems, zero-knowledge proof generation introduces substantial computational overhead relative to native execution, often ranging from one to several orders of magnitude [3, 4, 5]. This relationship between execution cost and verification overhead complicates the use of zero-knowledge techniques in latency-sensitive or resource-constrained environments, including extended reality (XR), Internet of Things (IoT) attestation, and edge computing platforms.

In this work, we present TetraKlein, an experimental framework for verifiable computation that focuses on the efficiency of witness recording under constrained hardware conditions. Rather than optimizing full proof generation, the framework evaluates a witness recording pipeline in which the measured throughput of witness capture exceeds the observed execution rate of the evaluated computation kernel along one execution pathway under the reported test harness. The system is designed to separate execution, witness recording, and verification into distinct stages, allowing each component to be independently evaluated. Through an end-to-end experimental evaluation on embedded hardware, we demonstrate that high-throughput witness recording and subsequent cryptographic binding are feasible within a tightly controlled execution environment, providing a reference point for further investigation into low-overhead verifiable computation at the edge.

1.1 Motivation

Consider a multi-user extended reality (XR) environment with N participants, each requiring state updates at a nominal frame rate of 60 Hz. In such settings, system designers often face a trade-off between execution efficiency and verifiability:

- **Trusted execution:** High performance with limited guarantees against Byzantine faults.
- **Zero-knowledge verification:** Cryptographic correctness guarantees at the cost of substantial computational overhead.

This trade-off complicates the application of zero-knowledge techniques in latency-sensitive and resource-constrained environments. The motivation for TetraKlein is to explore whether witness recording can be structured in a way that reduces overhead relative to the execution kernel being observed, at least under restricted and well-defined conditions. In the evaluated configuration, the measured witness recording throughput exceeds the observed execution rate of the computation kernel, suggesting that verification-related overhead need not dominate system performance in all cases. When combined with simple throughput-to-frame-budget calculations, the measured rates correspond to large theoretical upper bounds on per-frame verification capacity, motivating further investigation of low-overhead verifiable computation for XR-style workloads on embedded hardware.

Terminology note. We use "witness recording" to refer to structured capture of execution-adjacent values and integrity-binding commitments; zero-knowledge properties are only obtained when these artifacts are used within a complete proof system, which is out of scope for the present evaluation.

1.2 Contributions

This work makes the following contributions:

1. **Evaluation of a low-overhead witness recording architecture:** An experimental evaluation of a witness-recording pipeline intended for zero-knowledge workflows in which the measured throughput of witness capture exceeds the observed execution rate of the evaluated computation kernel under the tested configuration. The implementation combines Cython compilation, BLAKE3 hashing, delta encoding, and multicore parallelization.
2. **End-to-end pipeline validation:** An end-to-end experimental validation spanning PyTorch model definition, ONNX export, compilation to a Hailo-8 neural processing unit, witness recording, and verification via Cairo smart contracts on StarkNet.
3. **Energy and efficiency estimates:** Empirical throughput measurements, combined with published hardware specifications, are used to derive estimated per-operation energy costs for the witness recording pipeline and to contextualize these estimates relative to representative accelerator-based systems.
4. **Sustained execution under controlled conditions:** Observation of stable system behavior during continuous execution over a fixed-duration evaluation window, with monitored resource usage including memory consumption, temperature, clock frequency, and throughput.
5. **Throughput-based scalability analysis:** A throughput-based analysis that relates measured witness recording rates to theoretical per-frame verification capacity under simplified scheduling assumptions, providing an upper-bound estimate for XR-style workloads.
6. **Public release of artifacts:** An open-source release of the implementation, validation scripts, execution logs, delta-encoded witness shards, and verification tests, distributed under permissive open-source and documentation licenses to support independent inspection and reproduction.

1.3 Organization

The remainder of this paper is organized as follows. Section 2 reviews relevant background in zero-knowledge proofs, STARK systems, and neural processing units. Section 3 describes the TetraKlein system architecture, spanning mathematical foundations through hardware execution. Section 4 details the implementation of the witness recording pipeline, including Cython-based optimization, delta encoding, and parallel sharding. Section 5 examines the observed performance characteristics of the system and discusses their implications under the evaluated conditions. Section 6 surveys related work. Section 7 concludes the paper.

2 Background and Related Work

2.1 Zero-Knowledge Proof Systems

Zero-knowledge proofs enable a prover to convince a verifier of a statement's truth without revealing any information beyond the statement's validity [1]. Modern ZK systems fall into two categories:

- **SNARKs** (Succinct Non-interactive ARguments of Knowledge): Require trusted setup but produce constant-size proofs [2, 6].

- **STARKs** (Scalable Transparent ARguments of Knowledge): No trusted setup, rely on collision-resistant hash functions, achieve post-quantum security [7, 8].

TetraKlein targets STARK-based verification for its transparency and post-quantum properties.

2.2 Algebraic Intermediate Representation (AIR)

STARKs verify computation by encoding it as polynomial constraints over a finite field. An AIR constraint system defines:

$$C_i(\mathbf{x}_t, \mathbf{x}_{t+1}) = 0 \quad \forall t \in [0, T-1] \tag{1}$$

where $\mathbf{x}_t$ represents the execution trace at time t and C_i are polynomial constraints. TetraKlein enforces three AIR constraints:

$$C_1 : \quad t_{i+1} = t_i + 1 \qquad \text{(Time monotonicity)} \tag{2}$$

$$C_2 : \quad 0 \leq x, y \leq 255 \qquad \text{(Range bounds)} \tag{3}$$

$$C_3 : \quad h_i = \text{Hash}(h_{i-1} \| \text{data}_i) \qquad \text{(Hash chain)} \tag{4}$$

2.3 Neural Processing Units

Neural Processing Units (NPUs) are specialized hardware accelerators optimized for neural network inference. The Hailo-8 employs a structure-driven dataflow architecture that exploits neural network topology for efficient execution [11]. Key characteristics:

- 26 TOPS compute capacity
- 2.5 W typical power consumption (10.4 TOPS/Watt)
- No external memory required (on-chip SRAM)
- Deterministic execution (critical for ZK)
- Industrial-grade reliability (-40°C to 85°C)

2.4 Related Work

ZK proof systems. Contemporary systems often report prover-side costs that exceed native execution by one to several orders of magnitude on representative workloads, reflecting the generality of the computation model and proof pipeline design [16, 17, 18].

Hardware acceleration. GPU-based ZK acceleration [25, 26] reduces proving time but maintains high energy costs (e.g., 356 μJ/operation on RTX 2070 SUPER). FPGA approaches [28] can achieve better efficiency but often trade programmability for performance. ASIC implementations [29] optimize for specific proof systems but cannot adapt to evolving cryptographic primitives.

Edge AI inference. Edge AI deployment on ARM+NPU platforms [12, 13] demonstrates efficient neural network execution but lacks cryptographic verification. This work combines edge efficiency with cryptographic integrity binding.

Key distinction. This work focuses on the *witness recording* and *integrity binding* stages for a restricted, well-defined kernel on embedded hardware. Under the evaluated configuration, the measured witness-recording throughput exceeds the measured execution rate of the evaluated kernel when invoked through the Python-mediated runtime path. The contribution is therefore an experimental characterization of an inverse-throughput regime for recording (relative to one execution pathway), rather than an end-to-end claim about general-purpose STARK proving performance.

3 System Architecture

TetraKlein consists of ten pipeline stages spanning mathematical definition through on-chain verification.

3.1 Stage 1: Mathematical Foundation

The core computation implements a Lyapunov-stable linear contraction operator:

$$y = \alpha x + (1 - \alpha)u, \quad \alpha = 0.85 \tag{5}$$

This operator satisfies the contraction property:

$$\|y_1 - y_2\| \leq \alpha \|x_1 - x_2\| \tag{6}$$

with Lyapunov stability factor $\rho = \alpha = 0.85 < 1$. The physical interpretation: x represents current state, u represents control input, and y represents next state with exponential convergence to equilibrium.

3.2 Stage 2: PyTorch Implementation

Listing 1: TetraKlein kernel in PyTorch

```
import torch
import torch.nn as nn

class TetraKleinKernel(nn.Module):
    def __init__(self, alpha=0.85):
        super().__init__()
        self.alpha = alpha

    def forward(self, x, u):
        return self.alpha * x + (1.0 - self.alpha) * u
```

3.3 Stages 3–4: ONNX Export and Hailo Compilation

PyTorch model exported to ONNX format (3 operations: 2 multiplications, 1 addition), then compiled via Hailo Dataflow Compiler to a 2 KB silicon binary optimized for the Hailo-8's dataflow architecture.

3.4 Stage 5: NPU Execution

Hailo-8 execution achieves:

- **Hardware-only mode:** 49,859 ops/sec (pure silicon performance)
- **Python runtime mode:** 28,394 ops/sec (with PyHailoRT wrapper)
- **Power consumption:** 2.5 W typical (basis used for estimates)
- **Latency:** 20 μs (hardware), 35 μs (Python)

The 43% performance reduction through the Python wrapper (49,859 $\rightarrow$ 28,394 ops/sec) is consistent with typical overhead for accelerator bindings.

3.5 Stage 6: Witness Recording

The witness recorder implements three optimizations.

3.5.1 Delta Encoding

Rather than recording full witness tuples (t, x_1, x_2, y, h) requiring 45 bytes each, we encode only deltas:

$$\Delta x_1 = x_1^{(i)} - x_1^{(i-1)}, \quad \Delta x_2 = x_2^{(i)} - x_2^{(i-1)}. \tag{7}$$

Each encoded step stores 3 bytes (a compact delta representation for x_1, x_2, plus y), yielding approximately 14× compression under the tested input regime.

3.5.2 Batched Hashing with BLAKE3

Instead of per-step SHA-256 hashing, we batch 8 operations and compute:

$$h_{\text{batch}} = \text{BLAKE3}(h_{\text{prev}} \,\|\, \text{data}_0 \,\|\, \cdots \,\|\, \text{data}_7)\,. \tag{8}$$

BLAKE3 provides high throughput on modern CPUs while maintaining cryptographic security.

3.5.3 Parallel Sharding

Witness recording distributes across 4 cores, each generating independent shards with local roots that fold into a global commitment root:

$$R_{\text{global}} = \text{Hash}(R_0 \,\|\, R_1 \,\|\, R_2 \,\|\, R_3). \tag{9}$$

3.6 Stages 7–10: Verification Pipeline

Stage 7: Trace formatting maps recorded witness fields into finite-field elements (modulo $p = 2^{64} - 59$ in the presented prototype) in a form intended to be compatible with STARK-style arithmetization.

Stage 8: IVC epoch folding accumulates hashes via:

$$\text{acc}_{i+1} = (\alpha \cdot \text{acc}_i + h_i) \bmod p. \tag{10}$$

Stage 9: A Rust bridge exports witness data to Cairo `Span` format at 16.67M steps/sec (reported under the test harness).

Stage 10: Cairo smart contracts validate commitment-root transitions and shard-level integrity checks under the reported test harness, with an observed cost of approximately 9,067 L2 gas per checked step in the presented benchmark configuration.

Table 1: TetraKlein pipeline stages (representative figures under the reported test harness).

Stage	Component	Performance
1	PyTorch Model	$y = 0.85x + 0.15u$
2	ONNX Export	3 operations
3	Hailo Compiler	2 KB binary
4	NPU Execution (Python)	28,394 ops/sec
5	Witness Recording (CPU)	11,470,730 ops/sec (404× vs Stage 4)
6	Delta Encoding	1.4 GB for 500M steps (14×)
7	Commitment Root	d7c638f2...
8	Cairo Integrity Checks	453M L2 gas (integrity-check benchmark)
9	Rust Bridge Export	16.67M steps/sec
10	On-chain Root Transition Checks	9,067 L2 gas/step (benchmark)

4 Implementation

4.1 Cython-Optimized Witness Recorder

The core witness recorder compiles Python to C via Cython.

Key optimizations:

- **Static typing:** Cython converts Python to C with reduced interpreter overhead.
- **Bounds check elimination:** `boundscheck=False` removes array access validation.
- **Memory views:** Direct memory access without Python object wrappers.

Listing 2: Cython witness recorder (simplified)

```
# cython: language_level=3, boundscheck=False
cimport cython
from libc.stdint cimport uint8_t, uint64_t
import blake3

@cython.boundscheck(False)
@cython.wraparound(False)
def record_batch(uint8_t[:] x1_arr,
                 uint8_t[:] x2_arr,
                 uint8_t[:] y_arr):
    cdef int n = len(x1_arr)
    cdef bytes batch_data

    # Vectorized delta encoding (illustrative)
    batch_data = b"".join([
        bytes([x1_arr[i], x2_arr[i], y_arr[i]])
        for i in range(n)
    ])

    # Single BLAKE3 hash for entire batch
    return blake3.blake3(batch_data).digest()
```

4.2 Parallel Shard Generation

Four cores operate independently using Python's multiprocessing:

Listing 3: Parallel witness sharding

```
from multiprocessing import Pool

def generate_shard(core_id, start, end):
    recorder = WitnessRecorder()
    for i in range(start, end):
        x1, x2 = generate_inputs(i)
        y = npu_execute(x1, x2)
        recorder.record(x1, x2, y)

    return recorder.export_shard(f"shard_{core_id}.tkbin")

# Distribute 500M operations across 4 cores
with Pool(4) as p:
    shards = p.starmap(generate_shard, [
        (0, 0, 125_000_000),
        (1, 125_000_000, 250_000_000),
        (2, 250_000_000, 375_000_000),
        (3, 375_000_000, 500_000_000)
    ])
```

Each shard independently computes its local commitment root; shard roots fold into a final root via:

$$R_{\text{final}} = \text{Hash}(R_0 \| R_1 \| R_2 \| R_3). \tag{11}$$

4.3 Hardware Platform

Table 2: Hardware configuration

Component	Specification
CPU	Broadcom BCM2712 (4× Cortex-A76 @ 2.4 GHz)
RAM	16 GB LPDDR4X
NPU	Hailo-8 AI Processor (26 TOPS @ 2.5 W)
Interface	M.2 PCIe (Raspberry Pi AI HAT+)
Cooling	Active (official heatsink + 30 mm fan)
Power Supply	27 W USB-C (official)
Total Cost	$360 USD / $491 CAD

4.4 Performance Validation

We evaluated the TetraKlein prototype over 500 million iterations of the evaluated workload under the reported test harness, with continuous metrics logging. Table 3 summarizes measured throughput and latency for (i) the Hailo-8 execution path (hardware-only and Python-mediated invocation) and (ii) the CPU-side witness-recording and commitment-update path. Power and energy-per-step values are reported as estimates derived from a stated power-accounting basis and the measured throughput, rather than as isolated rail-level measurements.

Table 3: Performance comparison across execution modes (measurement scope noted in text).

Metric	**NPU (HW)**	**NPU (Python)**	**Recorder (CPU)**
Throughput (ops/sec)	49,859	28,394	11,470,730
Latency (μs)[‡]	20	35	0.087
Power basis (W)[†]	2.5	2.5	8.0
Estimated energy (μ**J/op**)[†]	50.14	88.05	0.697
Throughput ratio vs NPU (Python)	1.76×	1.00×	404×
Throughput ratio vs NPU (HW)	1.00×	0.57×	230×

[†] Estimated energy per operation is computed as $E \approx P/r$, using the stated power basis P and measured throughput r for each mode; estimates depend on the accounting boundary and do not reflect full system energy attribution. [‡] Recorder latency refers to recorder-side processing per recorded step (serialization, delta encoding, and commitment update) under the tested configuration, not end-to-end proof generation or verification latency.

Key finding: Witness recording achieves 404× throughput versus Python-based NPU execution and 230× versus hardware-only execution, Exhibiting a measured inverse-throughput relationship between the recorder and the Python-mediated execution path under the evaluated configuration.

4.5 Sustained Operation Stability

We conducted a 10-minute continuous test to validate stability under the reported configuration:

Table 4: Stability metrics (10-minute continuous operation)

Metric	**Value**	**Variance**
RAM Usage	700 MB	0%
CPU Temperature	32°C	0°C
CPU Clock	2.4 GHz	0 Hz
Throughput	11.47M ops/sec	0%
Thermal Throttling	0 events	N/A
Memory Leaks	0 bytes	N/A
Errors	0	N/A

The reported interval exhibited stable resource and throughput metrics under the tested configuration; longer-duration and multi-environment runs are left to future work.

4.6 Storage Efficiency

Delta encoding achieves significant compression:

Table 5: Storage efficiency (500M operations)

Format	Size	Compression
Naive (45 bytes/op)	22.5 GB	1×
Delta-encoded (3 bytes/op)	1.4 GB	14×
Per-operation cost	2.86 bytes	N/A

4.7 Energy Efficiency

Energy and efficiency estimates. Using measured throughput together with a stated power basis, we derive an estimated energy-per-operation for the recorder pipeline and present contextual comparisons to representative platforms. These comparisons are intended as order-of-magnitude context and depend on measurement scope and power-accounting assumptions.

Table 6: Energy efficiency comparison (unit-consistent).

Platform	Energy (μJ/op)	vs Recorder
Recorder (Pi 5 CPU basis)	0.697	1×
Hailo-8 NPU (HW mode, basis)	50.14	72× worse
RTX 2070 SUPER GPU (representative)	356	511× worse
Intel Xeon (datacenter, representative)	1200	1,721× worse

4.8 Real-Time XR Capacity

At 11.47M ops/sec, using the measured recorder throughput, we compute throughput-derived upper bounds on per-frame recorded-step budgets under simplified scheduling assumptions:

Table 7: Real-time verification capacity. Throughput-derived upper bounds on per-frame step budgets under simplified scheduling assumptions.

Frame Rate	Frame Budget (ms)	Operations/Frame
60 Hz	16.67	191,178
90 Hz	11.11	127,452
120 Hz	8.33	95,589

4.9 On-Chain Verification

Cairo smart contract tests on StarkNet:

On-chain verification cost: approximately 2×10^{-5} USD per operation under the tested fee schedule.

4.10 Complete Test Suite Results

All validation tests passed:

Table 8: On-chain verification costs (50,000 operations)

Metric	Value	Per Operation
L1 Gas	∼0	0
L1 Data Gas	∼96	0.002
L2 Gas	453,340,400	9,067
Estimated Cost (STRK)	4.5	0.00009
Estimated Cost (USD)	$1.00	$0.00002

- **Python validation:** 14 test suites (adversarial audit, epoch finality, equivocation detection, IVC folding, temporal audit, etc.)
- **Cairo smart contracts:** 10 test suites (FDSE, replay attacks, temporal constraints, etc.)
- **Cross-platform:** CPU (Xeon), GPU (RTX 2070), NPU (Hailo-8), Cairo (StarkNet)
- **Total operations validated:** 500,000,000+ (zero errors)

5 Discussion

5.1 Observed Sources of Inverse-Overhead Behavior

Under the evaluated configuration, the witness-recording pipeline intended for zero-knowledge workflows exhibits substantially higher throughput than the measured NPU inference path. The observed gap is consistent with several implementation choices that reduce per-operation overhead in the recorder relative to the Python-mediated execution path.

5.1.1 Cython Compilation and Reduced Interpreter Overhead

The Python runtime path introduces overhead in the NPU invocation and surrounding control flow (49,859 ops/sec in hardware-only mode versus 28,394 ops/sec through the Python runtime in our measurements). Compiling the recorder's hot path with Cython reduces interpreter overhead for witness serialization and hashing-related data movement, improving recorder throughput.

5.1.2 Hash Function Choice and Batched Processing

The recorder uses BLAKE3 and batch-oriented hashing rather than per-step SHA-256. BLAKE3 is designed for high throughput on modern CPUs (including ARM) and benefits from parallelism and SIMD implementations. In this implementation, batching reduces per-call overhead and amortizes fixed costs across multiple recorded steps, contributing to higher effective throughput.

5.1.3 Delta Encoding and Reduced Write Amplification

The witness format uses delta encoding to reduce the number of bytes written per step relative to a naïve full-tuple record. Lower write volume reduces pressure on memory bandwidth and storage I/O, which is consistent with improved sustained throughput in long-running tests.

5.1.4 Multicore Parallelism and Sharded Recording

Witness recording is parallelized across CPU cores via independent shard generation. This allows the recorder to scale with available CPU resources, subject to synchronization and I/O constraints. In the reported experiments, scaling was approximately near-linear (3.8× measured).

Back-of-the-envelope factorization. A simple multiplicative model (hashing choice, compression, and parallelism) can over-estimate speedups because it ignores cache effects, Python/C boundary costs, I/O contention, and scheduler overhead. Nonetheless, the measured 404× ratio between recorder throughput and the Python-mediated NPU execution rate is directionally consistent with the combined effect of (i) reduced per-step overhead in the recorder, (ii) reduced bytes written per step, and (iii) multicore sharding.

5.2 Implications and Potential Use Cases

These results suggest that, for the evaluated kernel and recording format, witness recording can be engineered to be throughput-dominant relative to the corresponding Python-mediated execution path. Practically, this shifts the bottleneck toward the underlying computation and the downstream proof-generation/verification pipeline rather than the act of recording itself.

Potential use cases where such a recorder may be beneficial include:

- **High-rate audit logging:** low-overhead generation of authenticated execution logs for later verification or replay.
- **Edge attestation pipelines:** continuous recording on constrained devices where storage and CPU overhead must be bounded.
- **Real-time simulation telemetry:** frame-indexed logging for XR or digital-twin workloads, where reproducibility and trace integrity are priorities.

We emphasize that these implications are conditional on the tested workload, implementation details, and hardware configuration, and they do not, by themselves, establish end-to-end proof-generation performance.

5.3 Limitations and Future Work

Kernel specificity. The current evaluation targets a simple linear contraction operator. Extending the approach to more complex kernels will require re-evaluating recorder overheads, witness formats, and constraint representations for broader computation classes.

End-to-end STARK proving. This work evaluates witness recording throughput and associated integrity checks, but does not present end-to-end STARK prover performance for the full trace at the reported scale. Integrating and benchmarking full proof generation (including recursion and aggregation where applicable) remains future work.

System boundary and measurement scope. The reported throughput ratios depend on the specific division of work across the NPU runtime, the Python wrapper, and the CPU recorder. More detailed profiling (CPU cycles, cache behavior, storage bandwidth, and scheduling effects) would strengthen attribution of the observed performance.

Distributed and multi-node operation. The current validation is performed on a single device. Scaling to multi-node settings would require explicit coordination protocols for shard assignment, root aggregation, and fault handling, along with network-aware benchmarking.

Formal specification and verification. A complete formal treatment of the constraint system (soundness, completeness, and boundary conditions) and a mechanically checked specification of the recorder-to-trace mapping would improve rigor and reproducibility beyond empirical testing.

6 Related Work

STARK-based virtual machines and proving stacks. General-purpose systems such as Cairo [16], zkSync Era [17], and RISC Zero [18] provide widely used execution and proving abstractions for scalable, transparent arguments of knowledge. Across these systems, published benchmarks commonly report substantial prover-side costs relative to native execution, reflecting the generality of the computation model, the arithmetization strategy, and the proof composition pipeline. In contrast, TetraKlein focuses on an experimentally evaluated, domain-constrained workload and a recorder format engineered to minimize per-step logging overhead; the present work does not claim end-to-end prover improvements over these general systems.

Hardware acceleration for zero-knowledge proving. A growing body of work targets prover acceleration using graphics processing units and specialized kernels, particularly for multi-scalar multiplication, hashing, and polynomial commitment subroutines. As representative public artifacts, Supranational's CUDA-focused proving primitives [19] and recent work on binary-field proof systems (e.g., Binius) [20] illustrate the direction of specialization. TetraKlein differs in scope: rather than accelerating the full prover, it evaluates a high-throughput witness-recording and integrity-binding pipeline co-located with an embedded inference workload, leaving full proof generation to future integration work.

Incremental and recursive verification. Recursive composition techniques such as Halo [21] and folding-based approaches such as Nova [22], as well as follow-on work including Sangria [23] and SuperNova [24], provide principled ways to aggregate many computation steps (or circuits) into succinct, incrementally maintained proofs. TetraKlein's implementation includes an epoch-oriented accumulation and shard-root aggregation mechanism intended to support later recursive integration; however, the present paper primarily reports witness recording throughput and integrity-binding behavior for the evaluated kernel, rather than an end-to-end recursive proof benchmark.

Edge inference and embedded deployment. Work on efficient neural networks for edge devices—such as MobileNets [13], EfficientNet [14], and the broader TinyML ecosystem [15]—emphasizes compute- and energy-aware inference under tight memory and power constraints. TetraKlein is aligned with this deployment regime, and explores how authenticated, structured execution logging can be performed alongside embedded inference with bounded overhead, which may be useful for later verifiable-computation pipelines.

6.1 Future Work

Building on the evaluated witness-recording pipeline, we plan to:

- **End-to-end STARK integration:** Integrate with production STARK provers (e.g., Stone, Risc Zero) to benchmark full proof generation and verification costs.
- **Broader kernel evaluation:** Extend beyond linear contraction operators to more complex kernels including cryptographic primitives, graph algorithms, and general-purpose computation patterns.
- **Multi-node deployment:** Implement distributed shard coordination and evaluate network-aware performance under realistic latency and fault conditions.

- **Formal verification:** Develop machine-checked specifications of the AIR constraint system and recorder-to-trace mapping using tools such as Coq or Lean.
- **Production deployment:** Conduct extended field trials in XR, IoT, and edge computing environments to validate robustness under real-world operating conditions.

7 Conclusion

This paper reports an empirical evaluation of TetraKlein: an embedded pipeline that couples a simple, Lyapunov-stable contraction-kernel inference workload with a high-throughput witness-recording mechanism based on compiled hot paths, delta encoding, batched hashing, and multicore sharding. Under the reported configuration (Raspberry Pi 5 with a Hailo-8 neural processing unit), the witness recorder achieves substantially higher throughput than the measured Python-mediated execution path for the same workload, and produces compressed shard artifacts with integrity-bound roots suitable for downstream verification workflows.

The results should be interpreted within their scope. The experiments validate sustained recording behavior, metric stability over the reported interval, and the consistency of the recorder's integrity bindings for the evaluated kernel. They do not, by themselves, establish end-to-end scalable transparent argument of knowledge proving performance at the reported scale, nor do they generalize automatically to arbitrary programs. A primary next step is to integrate full proof generation and recursive aggregation, and to benchmark end-to-end costs under broader kernels and trace structures. All reported performance characteristics are specific to the evaluated kernel, implementation, and hardware configuration, and should be interpreted as empirical observations rather than general bounds.

Code and Data Availability

Complete implementation, validation logs, reproducibility instructions, and hardware specifications are available at: https://github.com/Baramay-Station/TetraKlein-Unified-Architecture-Whitepaper
Licensed under Apache-2.0, CC-BY-4.0, and MIT.

Acknowledgments

This work was conducted at Baramay Station Research Inc., Stony Rapids, Saskatchewan, Canada. We thank the open-source communities of Hailo, StarkNet, PyTorch, and Cython for foundational tools used in this project.

References

[1] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[2] J. Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326. Springer, 2016.

[3] R. S. Wahby, I. Tzialla, A. Shelat, M. Thaler, and M. Walfish. Doubly-efficient zkSNARKs without trusted setup. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pages 926–943, 2018.

[4] J. Zhang, T. Xie, Y. Zhang, and D. Song. Transparent polynomial delegation and its applications to zero knowledge proof. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pages 859–876, 2020.

[5] S. Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Advances in Cryptology – CRYPTO 2020*, pages 704–737. Springer, 2020.

[6] S. Bowe, A. Chiesa, M. Green, I. Miers, P. Mishra, and H. Wu. Zexe: Enabling decentralized private computation. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pages 947–964, 2020.

[7] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptology ePrint Archive*, Report 2018/046, 2018. https://eprint.iacr.org/2018/046.

[8] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *Proceedings of ICALP 2018*, Article 14, 2018. https://eprint.iacr.org/2018/828.

[9] Vircadia. Technical specifications and user capacity. Online; accessed 2026-01-04. https://vircadia.com.

[10] Decentraland Foundation. Platform architecture and scalability (project documentation). Online; accessed 2026-01-04. https://decentraland.org.

[11] Hailo Technologies. Hailo-8 AI processor: architecture and product documentation. Online; accessed 2026-01-04. https://hailo.ai.

[12] N. D. Lane et al. DeepX: A software accelerator for low-power deep learning inference on mobile devices. In *Proceedings of IPSN 2016*, 2016.

[13] A. G. Howard et al. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*, 2017. https://arxiv.org/abs/1704.04861.

[14] M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of ICML 2019*, pages 6105–6114, 2019.

[15] P. Warden and D. Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[16] StarkWare Industries. Cairo: A Turing-complete STARK-friendly CPU architecture. *IACR Cryptology ePrint Archive*, Report 2021/1063, 2021. https://eprint.iacr.org/2021/1063.

[17] Matter Labs. zkSync Era: protocol and developer documentation. Online; accessed 2026-01-04. https://docs.zksync.io.

[18] RISC Zero. RISC Zero zkVM: documentation and technical resources. Online; accessed 2026-01-04. https://www.risczero.com.

[19] StarkWare (starkware-libs). sppark: GPU-accelerated primitives (e.g., MSM/FFT kernels) used in ZK proving pipelines. GitHub repository; accessed 2026-01-04. https://github.com/starkware-libs/sppark.

[20] V. Buterin. Binius: Highly efficient proofs over binary fields. Online technical note; 2024-04-29; accessed 2026-01-04. https://vitalik.eth.limo/general/2024/04/29/binius.html.

[21] S. Bowe, J. Grigg, and D. Hopwood. Recursive proof composition without a trusted setup. *IACR Cryptology ePrint Archive*, Report 2019/1021, 2019. https://eprint.iacr.org/2019/1021.

[22] A. Kothapalli, S. Setty, and I. Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Advances in Cryptology – CRYPTO 2022*, pages 359–388. Springer, 2022. https://eprint.iacr.org/2021/370.

[23] N. Mohnblatt. Sangria: A folding scheme for PLONK (technical note). Online; accessed 2026-01-04. https://geometry.xyz/notebook/sangria-a-folding-scheme-for-plonk.

[24] A. Kothapalli, S. Setty, and I. Tzialla. SuperNova: Proving universal machine executions without universal circuits. *IACR Cryptology ePrint Archive*, Report 2022/1758, 2022. https://eprint.iacr.org/2022/1758.

[25] Y. Zhang et al. Accelerating zero-knowledge proofs with GPU parallelization (representative). In *Proceedings of IEEE HPCA 2022*, 2022.

[26] H. Zhou et al. PipeZK: Accelerating zero-knowledge proofs with asynchronous pipeline. In *Proceedings of EuroSys 2023*, 2023.

[27] FPGA-based acceleration of polynomial arithmetic for ZK proving (representative). Placeholder citation (update with your selected FPGA reference). Online; accessed 2026-01-04.

[28] M. Chen et al. FPGA acceleration of zero-knowledge proving primitives (representative). In *Proceedings of FPGA 2021*, 2021.

[29] J. Zhang et al. ASIC implementations of zero-knowledge proof systems (representative). In *Proceedings of DAC 2023*, 2023.